# CS-6315: Advanced Software Engineering

## Complete Notes: Unit III and Unit IV

*For Test-2 (SCSIT, DAVV) - simple English, examples and diagrams*

| Unit | Topics covered (as per syllabus) |
|---|---|
| Unit III | Software change, Software evolution, Software maintenance: Models and Metrics, Reengineering, Reengineering Process and Activities, Program Comprehension, Reverse Engineering, Restructuring, Forward Engineering, Re-documentation, Software Aging |
| Unit IV (a) | Software Dependability, Safety, Availability, Reliability: Metrics, Approaches and Models. Software Quality, Quality Factors, Verification and Validation (V&V;), SQA |
| Unit IV (b) | Computer-Aided Software Engineering (CASE): Scope and Technology, CASE support in SDLC, Second Generation CASE Tools, Architecture of a CASE Environment |

**How to use these notes**

1. Every topic starts with a simple definition, then points, then a diagram or example.
2. Boxes: **blue** = definition (write this first in the exam), **green** = example, **red** = exam tip.
3. For a 20-mark question (usually 3-4 questions in the paper), write: Definition + Diagram + 5-6 points + Example. The last pages have a list of likely questions and a one-page quick revision.

# UNIT III: Software Change, Evolution and Maintenance

## 1. Software Change

**Definition**
Software change means modifying a software system after it is delivered, because users, business or technology keep changing. A change can be: **fixing a fault**, **adapting to a new environment** or **adding a new feature / moving to a new platform**.

- A software product is used for many years. During this time the customer's needs, laws, hardware and operating systems change, so the software must change too.
- Change is **inevitable** (it cannot be avoided) in every organisation.
- If we do not change software, it becomes less useful and less reliable. This is called **software aging** (see section 10).

**Example: Online railway ticket system**
Bug in payment page -> fix it (fault). Government introduces GST -> change the bill logic (new law). Users want UPI payment -> add a new feature. The whole system is moved from desktop to a mobile app (new platform). All of these are **software changes**.

## 2. Software Evolution and Lehman's Laws

**Software evolution**
The continuous process of changing and updating software during its life so that it stays useful in a changing world. **Evolvability** = capability of software to be evolved cost-effectively.

Meir Lehman said: software is a tool that models the real world. The real world keeps changing, so the software must change too. After studying large systems, he gave **eight laws**. They apply to **E-type systems** (systems that solve a real-world problem and live inside the real world, e.g. banking, railway, ERP).

| # | Law | Simple meaning |
|---|---|---|
| 1 | Continuing change | An E-type system must keep changing or it becomes less and less useful. |
| 2 | Increasing complexity | With every change, structure becomes more complex unless work is done to reduce it. |
| 3 | Self-regulation | Evolution of large systems follows a natural, statistically predictable pattern (size, bugs, release rate). |
| 4 | Conservation of organisational stability | Average work rate of the team stays almost constant; adding more people does not speed it up much. |
| 5 | Conservation of familiarity | Everyone (developers + users) must understand the system. Too big a release in one go causes trouble, so growth per release stays steady. |
| 6 | Continuing growth | Features must keep growing to keep users satisfied. |
| 7 | Declining quality | Quality seems to fall unless the system is rigorously maintained and adapted. |
| 8 | Feedback system | Evolution is a multi-level feedback process; feedback from users must be used to improve. |

**Exam tip**
Remember the laws with the phrase: **C-I-S-C-C-C-D-F** = Change, Increasing complexity, Self-regulation, Conservation (stability), Conservation (familiarity), Continuing growth, Declining quality, Feedback.

## 3. Software Maintenance

**Definition (IEEE)**
Software maintenance is the process of **modifying a software system or component after delivery** to correct faults, improve performance or other attributes, or adapt it to a changed environment.

- Maintenance starts **after** the product is delivered to the customer and continues until retirement.
- Maintenance takes **60-80% of the total life-cycle cost**. About 75-80% of that is spent on enhancement (new features), not on bug fixing.
- Pigoski: maintenance includes pre-delivery activities (planning for support) and post-delivery activities (modification, training, help desk).

### Why is maintenance needed?

- Defects found by users after release.
- Customer needs change (add / delete features).
- Operating environment, hardware, laws or company policy change.
- To avoid software aging and keep quality high.

## 4. Categories (Types) of Maintenance

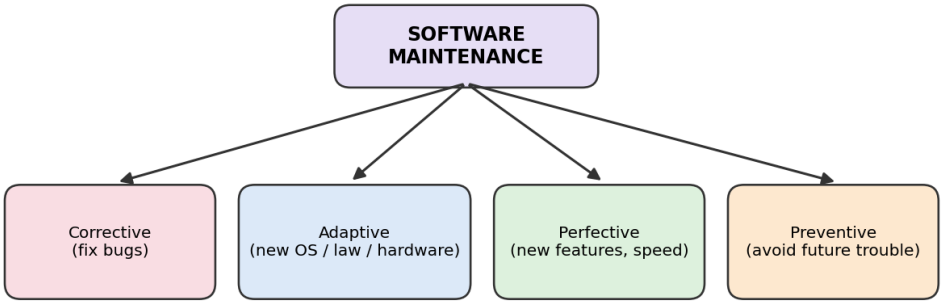



*Fig 1: Categories of maintenance*

| Type | Purpose | Example |
|---|---|---|
| Corrective | Fix defects / failures found after delivery | Fix a wrong interest calculation in a bank app |
| Adaptive | Change software for a new environment (OS, hardware, law, DB) | Update the app for a new Android version; add new GST rates |
| Perfective | Improve performance or maintainability, add new user requests (largest share) | Make search 3x faster; add a dark mode |
| Preventive | Update software in advance to avoid future problems | Clean up code, upgrade a library before it becomes unsupported |
| Emergency | Urgent, unplanned repair that needs immediate attention | Server crashes during exam registration - fix at once |

## 5. Maintenance Process Models

A maintenance process model organises maintenance into phases and defines their order. Models in the syllabus: **Quick-Fix, Osborne's, Iterative-Enhancement, Full-Reuse, IEEE 1219, ISO 12207**.

### 5.1 Quick-Fix Model

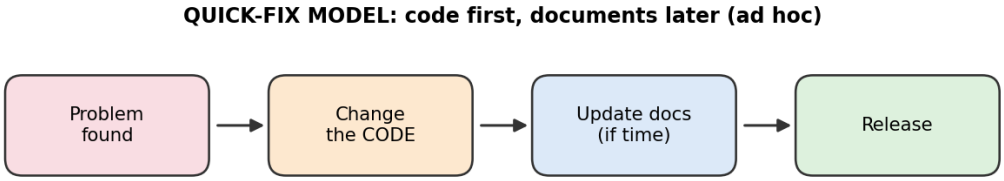



*Fig 2: Quick-fix model*

- An **ad hoc** approach: find the problem, change the code first, update documents later.
- Fast, used under time pressure.
- **Drawbacks:** no planning or impact analysis, documents become out of date, code structure gets worse, long-term cost increases.
- Good only for small, urgent fixes.

## 5.2 Osborne's Model

- Based on the reality of the maintenance environment. It says technical problems in maintenance come from **poor communication and poor management control**.
- Strategy: put maintenance requirements in the change specification, set up a quality assurance programme, and define metrics to check that maintenance goals are met.

## 5.3 Iterative-Enhancement Model

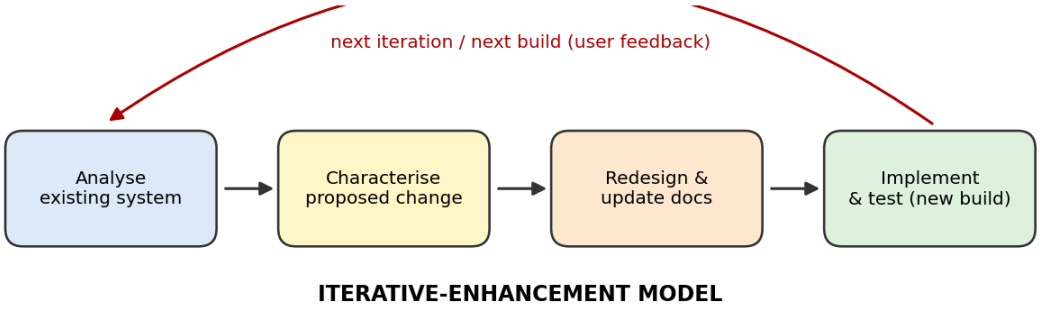



*Fig 3: Iterative-enhancement model*

- Maintenance is an **iterative process**. Requirements cannot be fully known in the beginning, so the system is changed in **builds**; each build improves the previous one using user feedback.
- Each iteration: analyse existing system -> characterise the change -> redesign and update documents -> implement and test.
- **Advantages:** documentation stays updated, system remains maintainable, changes are faster. **Limitation:** poor if complete documentation is not available.
- Best for long-life systems that evolve over time.

## 5.4 Full-Reuse Model

- Maintenance is treated as **reuse-oriented development**. Start with requirements and design of the new version and **reuse** requirements, design, code and tests from the old version stored in a **reuse repository**.
- Good for product lines and component-based systems or reengineering to a component platform. Needs an initial cost to build the reuse environment.

## 5.5 IEEE 1219 Model

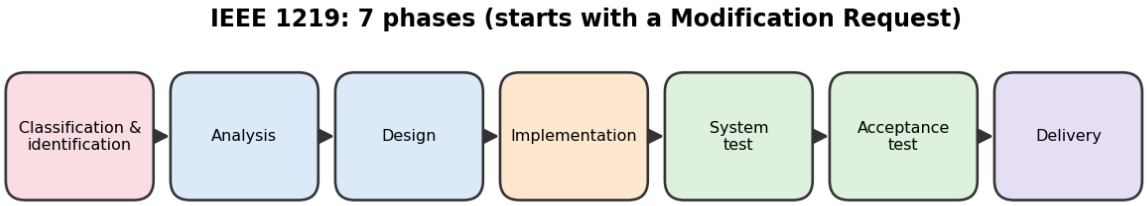



*Fig 4: IEEE 1219 maintenance process*

- Seven phases: **Classification and identification -> Analysis -> Design -> Implementation -> System test -> Acceptance test -> Delivery**.
- It begins with a **Modification Request (MR)** from a user, customer, programmer or maintenance manager.

### 5.6 ISO/IEC 12207 Model

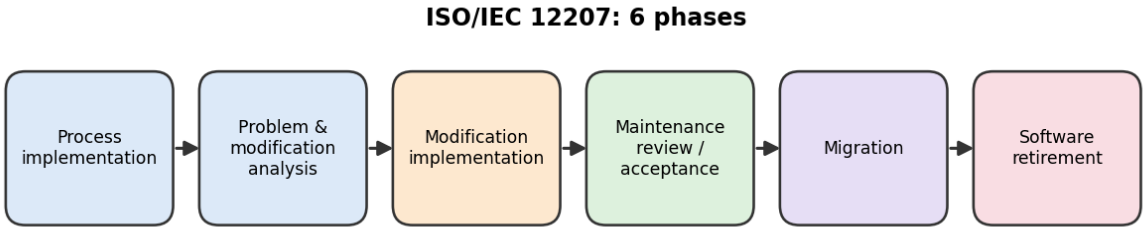



*Fig 5: ISO 12207 maintenance process*

- Six phases: **Process implementation -> Problem and modification analysis -> Modification implementation -> Maintenance review/acceptance -> Migration -> Software retirement**.
- Extra phases compared to IEEE 1219: **migration** (moving to a new environment) and **retirement** (shutting the system down).

| Model | Key idea | Best use |
|---|---|---|
| Quick-fix | Code first, docs later | Emergency small fix |
| Osborne | Fix communication and management control | Large organisations |
| Iterative enhancement | Change in builds with feedback | Long-life systems |
| Full reuse | Reuse parts from repository | Product lines, component systems |
| IEEE 1219 | 7 phases from modification request | Standard process |
| ISO 12207 | 6 phases incl. migration and retirement | Complete life cycle |

## 6. Maintenance Metrics and Cost

### 6.1 Annual Change Traffic (ACT) - Boehm

**ACT**
The fraction of a software product's source instructions that **change during a year** through addition, deletion or modification.

**ACT = ($KLOC_{added}$ + $KLOC_{deleted}$) / $KLOC_{total}$**
**Maintenance effort = ACT x Development effort**
With adjustment factors: **Maintenance effort = ACT x Development effort x EAF** (EAF = Effort Adjustment Factor, because every project is different).

**Example: From the lecture (Example 13.1)**
ACT = 9.5% per year, development cost = Rs. 56 lakh, maintenance for 7 years.
Total cost = 56 + 7 x (56 x 0.095) = 56 + 37.24 = **Rs. 93.24 lakh**.

**Example: Using KLOC**
Software size = 80 KLOC. In one year 5 KLOC added and 3 KLOC deleted. ACT = (5+3)/80 = **0.10 (10%)**. If development effort was 200 person-months, maintenance effort per year = 0.10 x 200 = **20 person-months**.

## 6.2 Software Maturity Index (SMI)

**SMI**
Measures the **stability (release readiness)** of a product when modules are added, changed or deleted between releases.

**SMI = [ $M_t$ - ($F_a$ + $F_c$ + $F_d$) ] / $M_t$**
$M_t$ = modules in the current release, $F_a$ = added, $F_c$ = changed, $F_d$ = deleted from the previous release. As **SMI approaches 1.0, the product is stabilising**.

**Example: SMI**
Current release has 100 modules; 6 added, 10 changed, 4 deleted. SMI = (100 - 20)/100 = **0.80**. Next release with fewer changes will give a value closer to 1, i.e. more stable.

**Exam tip**
In the slide the formula is written without brackets; the correct meaning is the whole sum (Fa+Fc+Fd) is subtracted from Mt, and then divided by Mt. Write it with brackets.

## 6.3 Other points on cost

- Maintenance cost is lower if defects are found in earlier phases of development.
- Cost differs from application to application.

# 7. Reengineering

## 7.1 Why reengineering? (Legacy systems)

A **legacy system** is an old but still important system. Symptoms: poor or missing documentation, complex code, cannot meet new customer needs, low business value, obsolete technology, high maintenance cost, frequent failures, constant need of patches, system feels outdated.

Such systems are too costly to maintain but too valuable to throw away. The solution is to **transform them onto a modern platform**: this is reengineering.

**Reengineering**
The process of **analysing and representing an existing (subject) system at a higher level of abstraction and reconstituting it into a new, improved form**.
**Reengineering = Reverse engineering + Δ (changes) + Forward engineering**

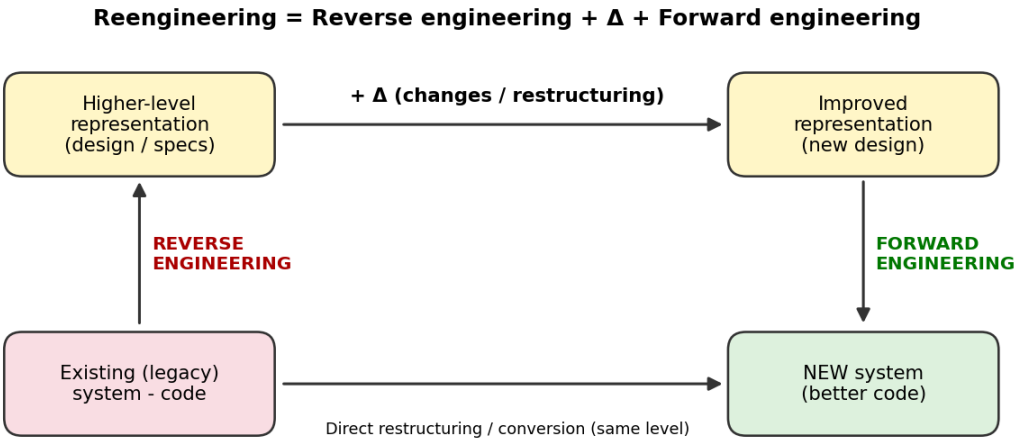



*Fig 6: Reengineering = reverse engineering + changes + forward engineering*

**Example: Old COBOL payroll system**
A company still runs payroll in COBOL with no documents. Step 1: study code and recover design (reverse engineering). Step 2: clean structure and change data model (restructuring, Δ). Step 3: build it in Java with a web interface (forward engineering). Business rules are kept; technology and structure are improved.

## 7.2 Reengineering process and activities

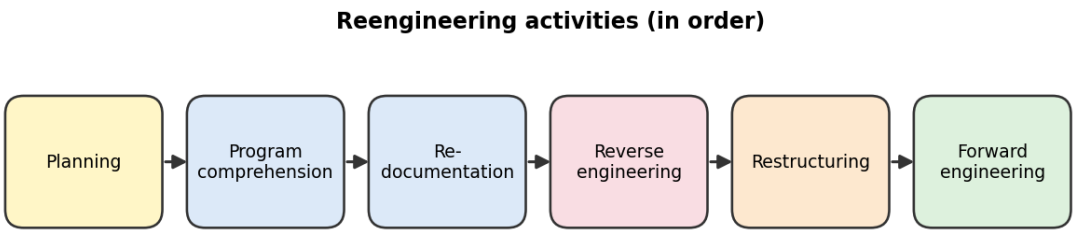



*Fig 7: Activities of the reengineering process*

The six activities in order: **Planning, Program comprehension (inventory analysis), Re-documentation, Reverse engineering, Restructuring, Forward engineering.**

# 8. Reengineering Activities in Detail

## 8.1 Planning for Reengineering

- Decide scope of work, estimate effort, duration and cost, staff and resources, risk management and change management.
- Analyse existing products for problems (documents, code quality, business value) and choose which systems to reengineer.
- Select tools and repositories.

## 8.2 Program Comprehension (Inventory Analysis)

- Process of **collecting and understanding the inventory** of the existing system: program files, documents, tools, human experts, file structures, development environment.
- Helps to understand control flow, data usage, program linking and prototypes.
- Often programmers know the **domain** but not the **code**; comprehension fills this gap. It is the most time-consuming part (often 50% of maintenance effort).

## 8.3 Re-documentation

- Creating a **semantically equivalent** description at the same level of abstraction: documents are updated, rewritten or replaced.
- Documents affected: requirement specification, design, design decision report, configuration, data dictionary, user and reference manuals, change document.
- Good documents improve the quality of future maintenance.

## 8.4 Reverse Engineering

**Reverse engineering**
Recovering the design specification of an existing system **from its implementation (code)** and representing it at a **higher level of abstraction**.

- Extracts data, architecture, design and content information from the old system.
- Produces alternative representations: **structure charts, data flow diagrams, ER diagrams, UML diagrams**.
- Example: from the source code of a library software, draw the ER diagram of its database and the class diagram.

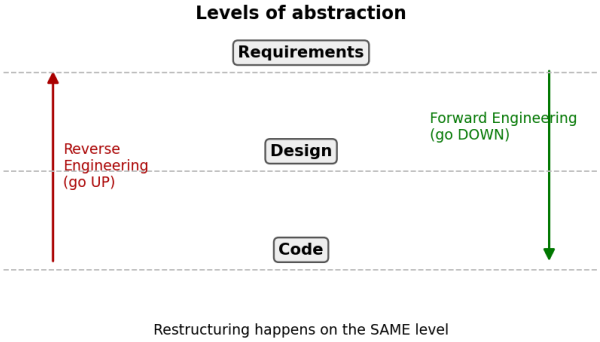



*Fig 8: Reverse engineering goes up, forward engineering goes down, restructuring is on the same level*

## 8.5 Restructuring

- Transforms a system from one representation to another **at the same level of abstraction**, keeping the **same functionality** (semantics).
- **Code restructuring:** remove GOTOs, break big functions, reduce complexity, improve structure. **Data restructuring:** clean up data items and structures, normalise the database.
- Result: code easier to understand, test and change.

**Example: Restructuring**
A 2000-line function with nested if-else is split into 10 small functions with meaningful names. Output of the program is the same, but it is far easier to maintain.

### 8.6 Forward Engineering

- The traditional move from **high-level design to physical implementation**.
- After reverse engineering recovers all artifacts, forward engineering uses design details (object models, use cases, pseudo code) to build the new system, e.g. in an object-oriented language.
- The new system may also have new features added at this step.

| Activity | Direction / level | Output |
|---|---|---|
| Reverse engineering | Code -> design (up) | DFD, ER, UML, structure charts |
| Restructuring | Same level | Cleaner code / data, same function |
| Re-documentation | Same level | Updated documents |
| Forward engineering | Design -> code (down) | New improved system |

## 9. Maintenance vs Reengineering

| Point | Maintenance | Reengineering |
|---|---|---|
| Scope | Small, local changes | Major change of the whole system |
| Goal | Keep system running / add feature | Improve structure, move to new platform |
| Cost | Low per change, high over time | High at start, saves cost later |
| When | Daily, as needed | When system becomes too costly to maintain (legacy) |

## 10. Software Aging

**Software aging**
A phenomenon in which running or repeatedly maintained software shows **performance degradation and/or an increasing failure rate** over time.

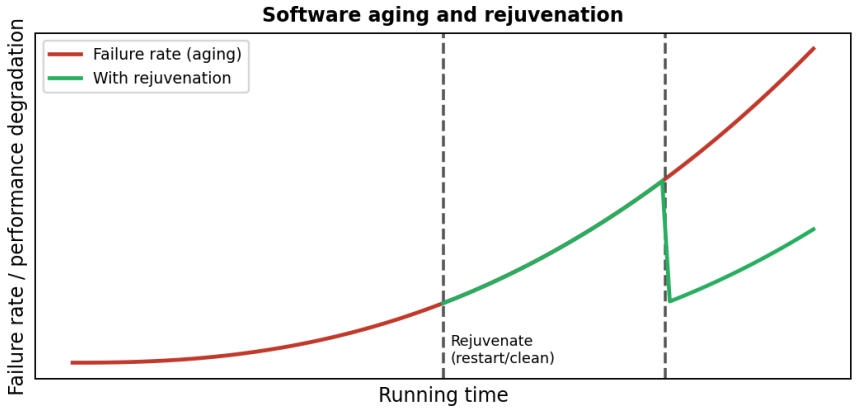



*Fig 9: Failure rate grows with aging; rejuvenation brings it down*

### Causes

- Continuous change and patching damages the structure.
- Size and complexity grow with every upgrade.
- Poor or ignored documentation after changes.
- Resource leaks (memory leak, unclosed files, growing swap use) and accumulated errors.
- Developers' lack of understanding; software cannot adapt to new platforms.

### Symptoms

- Hard to satisfy business needs, slow performance, more bugs and failures, uses more resources, cannot run on new platforms.

### Prevention / cure

- **Design for change** (modular, low coupling), proper documentation, **preventive maintenance**, keeping structure clean, fault-tolerance techniques.
- **Software rejuvenation:** periodically stop the running software, clean its internal state (free memory, clear files, restart) and restart it. It is a cost-effective way to remove aging effects.
- **Aging-related failure:** a fault that shows only after a long run time (e.g. a memory allocation finally fails and the process crashes).

**Example: Aging**
A web server runs 90 days without restart; a small memory leak grows until it crashes. Restarting the server every Sunday night (rejuvenation) avoids the crash.

**Exam tip**
For a 20-mark question on Unit III: (1) define maintenance and its types, (2) draw 2 process models (IEEE 1219, iterative), (3) ACT/SMI formulas with example, (4) reengineering diagram with 6 activities, (5) software aging.

# UNIT IV (a): Dependability, Safety, Availability and Reliability

## 1. Software Dependability

**Dependability**
The extent to which a (critical) system is **trusted** by its users: the user's confidence that it will work as expected and will not fail in normal use.

- For critical systems (aircraft control, medical devices, banking) dependability is the **most important** property.
- Usefulness and trustworthiness are not the same: a system can be useful without being trusted, but untrustworthy systems are often rejected by users.

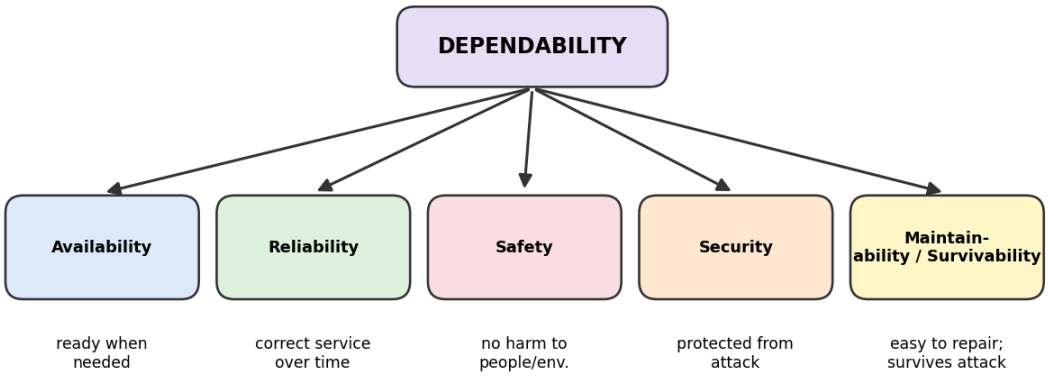



*Fig 10: Dimensions of dependability*

| Dimension | Meaning (simple) |
|---|---|
| Availability | Probability that the system is up and can deliver service when requested |
| Reliability | Probability of failure-free operation for a given time in a given environment |
| Safety | Ability to operate without causing injury, death or damage to the environment |
| Security | Ability to protect itself from accidental or deliberate attack |
| Maintainability | Ease of repairing the system after failure or adding features (a static attribute) |
| Survivability | Ability to keep delivering services during an attack or component failures (includes resilience) |

### Cost of dependability

- Cost rises **exponentially** as we go from low -> medium -> high -> ultra-high dependability, because we need more expensive techniques/hardware and much more testing and validation.
- Sometimes it is cheaper to accept a less dependable system and pay for failures; but untrustworthy systems damage reputation and may lose data. Business systems may need only moderate dependability; safety-critical systems need very high.

## 2. Software Availability

**Availability**
The probability that the system, **at a given point in time**, is operational and able to deliver the requested services.

- Availability takes **repair time** into account: a system that fails often but is repaired in seconds can still have high availability (e.g. a telephone exchange).
- **Availability = MTTF / (MTTF + MTTR) x 100%** (= Uptime / Total time).
- Important for servers, telecom, banking, online exams.

**Example: Availability**
A server runs 90 hours on average before failing (MTTF = 90 h) and takes 10 hours to repair (MTTR = 10 h). Availability = 90/(90+10) = **90%**. If MTTR is reduced to 1 hour, availability = 90/91 = **98.9%**.

## 3. Software Reliability

**Reliability (IEEE)**
The **probability of failure-free operation** of software over a given time interval and under given conditions.

- Depends on the presence or absence of defects. A system's reliability = reliability of hardware + software together.
- Reliability is measured in the **operational environment**; different users use the system differently, so it is not a fixed property.

### 3.1 Fault, Error, Failure (terminology)

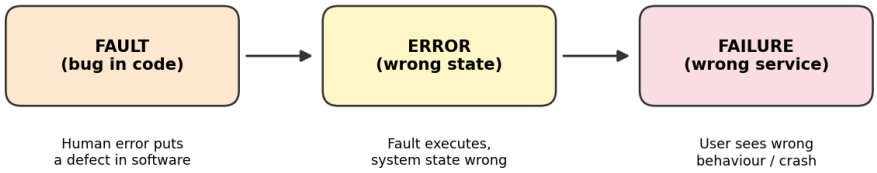



*Fig 11: Fault -> Error -> Failure chain*

| Term | Meaning | Example |
|---|---|---|
| Human error / mistake | Wrong action by a developer | Programmer forgets to check divide-by-zero |
| Fault (defect, bug) | Incorrect part of the code | Missing check in the code |
| Error | Wrong system state at run time | Variable holds 0 and division is attempted |
| Failure | System gives wrong or no service to the user | Program crashes |

- Not every fault causes an error (code may never run), and not every error causes a failure (built-in recovery may fix it).

### 3.2 Hardware vs Software reliability

| Point | Hardware | Software |
|---|---|---|
| Cause of failure | Wear, ageing, physical damage | Design / logic faults present from the start |
| Failure pattern | Bathtub curve (early failures, stable period, wear-out) | Fails due to bugs; reliability improves as bugs are removed (but aging can raise failure rate) |
| Repair | Replace the part | Fix the code (no physical replacement) |
| Redundancy | Identical copies help | Identical copies do NOT help (same bug); needs design diversity |
| Environment | Temperature, vibration matter | Input data and usage pattern matter |

### 3.3 Reliability Metrics

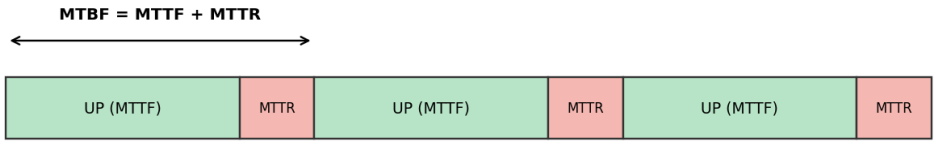



*Fig 12: MTTF, MTTR and MTBF on a timeline*

| Metric | Meaning | Use / Example |
|---|---|---|
| POFOD - Probability of Failure On Demand | Likelihood that a service request fails. = failed requests / total requests | Systems with rare requests, e.g. a safety shutdown system. 2 failures in 1000 demands -> POFOD = 0.002 |
| ROCOF - Rate of Occurrence of Failure | Number of failures per unit time (or per transactions) | Systems used continuously. 2 failures in 1000 h -> ROCOF = 0.002 / hour |
| MTTF - Mean Time To Failure | Average time between two consecutive failures (running time only) | ROCOF = 1/MTTF. MTTF = 500 h |
| MTTR - Mean Time To Repair | Average time to detect and fix a failure | MTTR = 5 h |
| MTBF - Mean Time Between Failures | **MTBF = MTTF + MTTR** | MTBF = 500 + 5 = 505 h |
| Availability | Probability system is available. = MTTF/(MTTF+MTTR) | 500/505 = 99.0% |

## 3.4 Approaches to achieve reliability

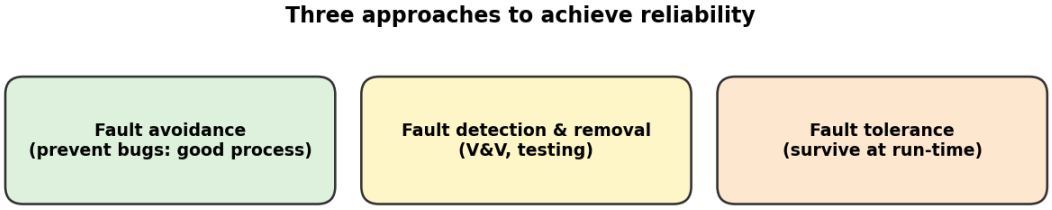



*Fig 13: Fault avoidance, detection and tolerance*

- **Fault avoidance:** use development techniques that reduce mistakes: good requirements, formal methods, structured programming, reviews, CASE tools, strong typing.
- **Fault detection and removal:** Verification and validation: inspections, testing, static analysis, to find and fix faults before release.
- **Fault tolerance:** keep working even when faults occur: error detection, damage assessment, recovery, **redundancy** (N-version programming, recovery blocks, backups, exception handling).

**Example: Fault tolerance**
Aircraft flight control runs three independently written versions of software; if one gives a different answer, the other two outvote it (**N-version programming**).

## 3.5 Reliability Models

**(a) Input-output mapping model (from lecture).** Think of the program as a mapping from the set of all inputs to outputs. Some inputs cause **erroneous outputs**. Reliability = probability that a given input is *not* in that erroneous set. Different users use different inputs (**operational profile**), so reliability depends on usage.

**(b) Reliability growth models** (used during testing, as faults are found and removed):

- **Jelinski-Moranda model:** failure rate is proportional to the number of faults remaining; each fix removes one fault perfectly, so failure rate drops by the same step after every fix.
- **Basic execution time model (Musa):** failure intensity falls exponentially with the number of failures experienced / execution time.
- **Littlewood-Verrall model:** a Bayesian model; fixing a fault may not always help, so the size of improvement varies.
- **Operational profile testing:** test using the same pattern of inputs real users will use, to get a realistic reliability estimate.

### 3.6 Reliability improvement - important facts

- Removing X% of faults does **not** improve reliability by X%. An IBM study found that removing 60% of defects improved reliability by only 3%, because many faults sit in rarely executed code.
- A program with known faults can still be seen as reliable if users never hit them. Users give more weight to serious failures than small inconveniences.

## 4. Software Safety

**Safety**
The ability of a system to operate, normally or abnormally, **without causing human injury or death or damage to its environment**.

- **Primary safety-critical systems:** embedded software whose failure directly threatens people (airbag controller, pacemaker, railway signalling).
- **Secondary safety-critical systems:** failure causes faults in other systems that threaten people (e.g. a hospital patient database giving a wrong blood group).
- Safety requirements are **exclusive requirements**: they say what must NOT happen.

### Safety vs Reliability

| Reliability | Safety |
|---|---|
| Conformance to the specification | Never cause harm, whether or not the spec is followed |
| Failure = deviation from spec | Hazard/accident = harm to people or environment |
| Necessary for safety but not sufficient | A reliable system can still be unsafe (wrong specification, wrong command at the wrong time) |

### Safety terminology

- **Accident (mishap):** unplanned event causing death, injury or damage. **Hazard:** condition that can lead to an accident. **Damage:** loss from an accident. **Hazard severity / probability / risk:** how bad / how likely / combination.

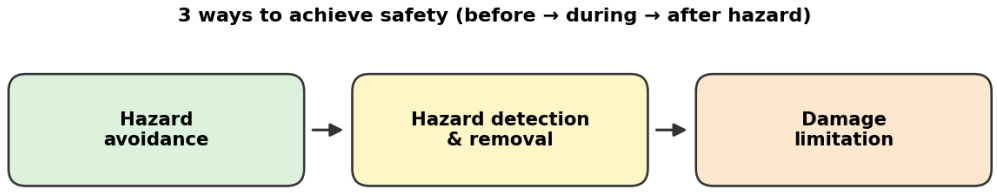



*Fig 14: Three ways to achieve safety*

- **Hazard avoidance:** design so that some hazards cannot occur (e.g. two-hand control so an operator's hand can't be in the machine).

- **Hazard detection and removal:** detect and remove hazards before an accident (e.g. pressure sensor opens relief valve).
- **Damage limitation:** protective features reduce damage (e.g. airbag, automatic shutdown, fire doors).

**Normal accidents:** in complex systems accidents seldom have a single cause; they come from combinations of small failures. Since all combinations cannot be predicted, absolute safety is impossible.

## 5. Security (relation with dependability)

- Security = ability to protect the system from accidental or deliberate external attack. It is a pre-requisite for availability, reliability and safety; if a networked system is insecure, claims about its reliability and safety are invalid.
- **Damage from insecurity:** denial of service, corruption of programs/data, disclosure of confidential information.
- **Security assurance:** vulnerability avoidance, attack detection and elimination, exposure limitation (e.g. backups).

**Exam tip**
Common 5-mark: 'Differentiate reliability, availability, safety'. Reliability = works without failure; Availability = ready when needed (includes repair time); Safety = does not cause harm.

# UNIT IV (b): Software Quality, V&V; and SQA

## 6. Software Quality

**Software quality**
The degree to which software **meets customer requirements**, has the required attributes with good values, follows standards and has a sound internal design, within an acceptable cost and schedule.

- **External quality** (what the user sees: speed, ease of use) and **internal quality** (design, code structure). Good external quality does not guarantee good internal quality, just like a building with a beautiful look can have weak foundations.
- A standard, well-defined **process** leads to a good **product**. Quality is also about cost and delivery date.

## 7. Software Quality Factors (McCall, Richards, Walters)

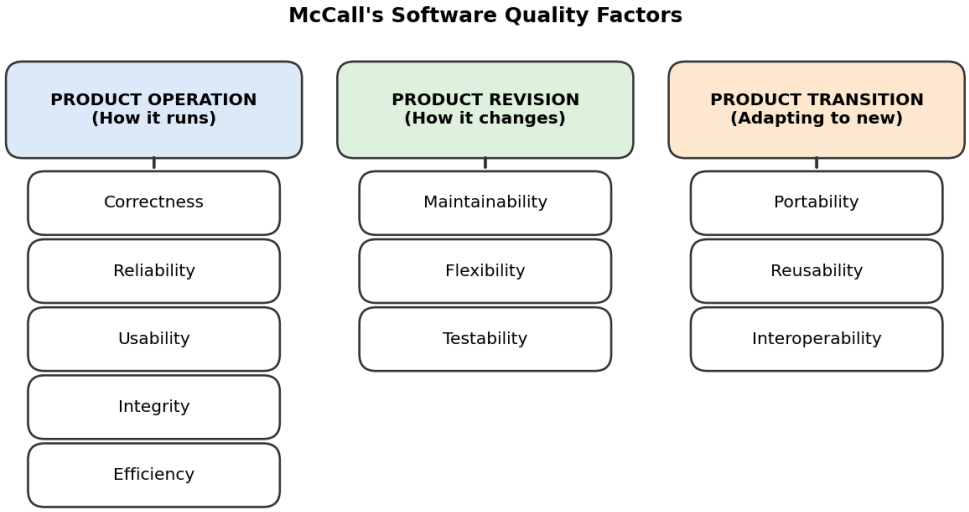



*Fig 15: McCall's quality factors - 11 factors in 3 groups*

| Factor | Meaning (simple) |
|---|---|
| Correctness | Does exactly what the specification says |
| Reliability | Performs the intended function without failure for a specified period |
| Usability | Effort needed to learn, operate and use it |
| Integrity | Control of unauthorised access to data and programs |
| Efficiency | Computing resources used (CPU time, memory, bandwidth) |
| Maintainability | Ease of finding and fixing errors (mean time to change) |
| Flexibility | Cost/effort to modify an operational program |
| Testability | Effort needed to test that it works as intended |
| Portability | Effort to move to another hardware/software environment |
| Reusability | Extent to which parts can be reused in other software |
| Interoperability | Effort needed to couple it with another system |

## 8. Verification and Validation (V&V;)

**V&V;**
The process of checking that a software product **meets its specification (verification)** and **satisfies the user's real needs (validation)**. Goal: confidence that the software is 'fit for purpose'.

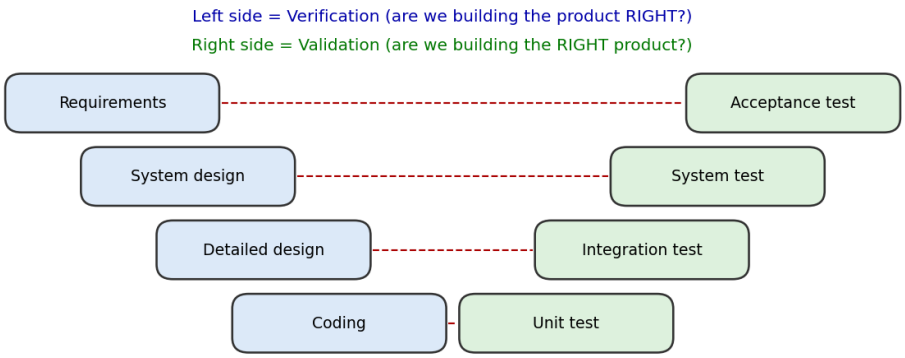



*Fig 16: V-model showing verification and validation*

| Point | Verification | Validation |
|---|---|---|
| Question (Boehm) | Are we building the product RIGHT? | Are we building the RIGHT product? |
| Checks | Work products against specification | Final software against user needs |
| When | During each development phase | During or at the end of development |
| Methods | Reviews, walkthroughs, inspections, static analysis (no execution) | Testing: unit, integration, system, acceptance (execution) |
| Done by | Developers / QA team | Testers and the customer |
| Example | Checking the design document matches the SRS | Letting the college use the exam software and confirm it is what they wanted |

## 9. Cost of Quality

- **Cost of conformance** (spent to achieve quality): **Prevention cost** (planning, training, formal reviews, test equipment) + **Appraisal cost** (inspection, testing, calibration).
- **Cost of non-conformance: Failure cost**: internal (rework, retest before delivery) and external (complaints, warranty, bug fixes after delivery, lost customers).
- Philip Crosby (1979): **'Quality is free'**: preventing defects costs less than fixing them later. Defects found late cost 10-100 times more.

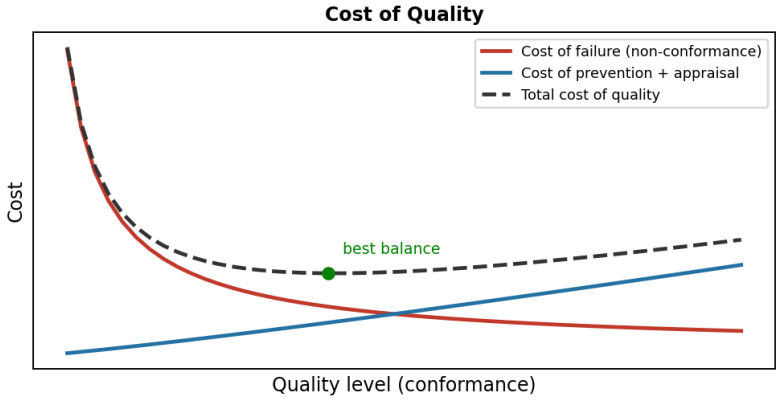



*Fig 17: Prevention cost goes up, failure cost goes down; best balance in the middle*

## 10. Software Quality Assurance (SQA)

**SQA (IEEE)**
A **planned and systematic pattern of all actions** necessary to provide adequate confidence that a product conforms to established technical requirements. It is a **preventive** approach (prevent defects), unlike testing which detects defects.

### SQA activities

- **By the software development team:** apply technical methods and tools, conduct V&V; (reviews), perform measurements, perform testing.
- **By the SQA team:** prepare the SQA plan, participate in development process, review engineering activities for process compliance, software audits, record keeping, reporting.

### Quality Assurance vs Quality Control

| Quality Assurance (QA) | Quality Control (QC) |
|---|---|
| Process-oriented, preventive | Product-oriented, detective |
| Ensures we follow the right process | Checks the product has no defects |
| Done throughout the life cycle | Done after a product is built (testing) |

## 11. ISO 9000 Quality Standard

- ISO (International Organization for Standardization) gives international standards. **ISO 9000** is a family of quality management standards, applicable to all industries; for software, **ISO 9001** and guideline ISO 90003 are used. It certifies the **process**, not the product.

### Eight quality management principles

- Customer focus; Leadership; Involvement of people; Process approach; System approach to management; Continual improvement; Factual approach to decision making; Mutually beneficial supplier relationships.

### ISO 9000 certification steps

- Proposal stage -> Pre-assessment -> Document review and adequacy audit -> Compliance audit -> Registration -> Continued surveillance.

| Advantages | Disadvantages |
|---|---|
| Improves procedures and reduces cost; good marketing tool; competitive advantage; ensures safe, reliable products | Change can be costly; training needed; certifies the process but does not guarantee high quality; some customers may prefer non-ISO vendors unfairly |

## 12. Capability Maturity Model (SEI-CMM)

**CMM**
A model by the **Software Engineering Institute (SEI), Carnegie Mellon University (1986-87)** to measure the **maturity of an organisation's software process** and give a path to improve it, in **five levels**. Each level (except 1) has **Key Process Areas (KPAs)** that must be satisfied.

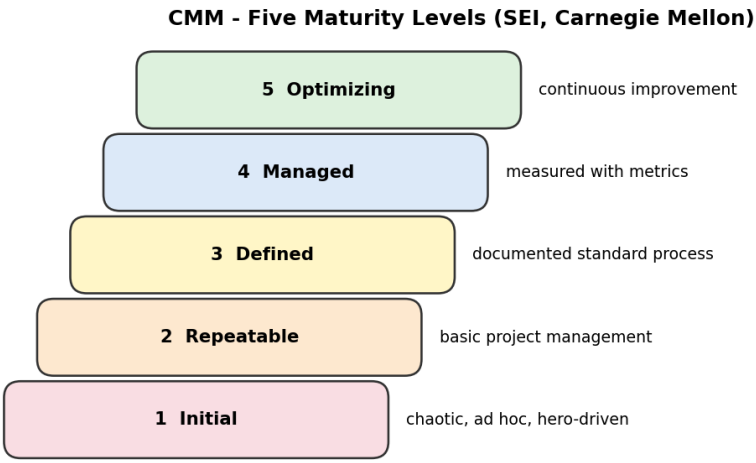



*Fig 18: Five levels of CMM*

| Level | Characteristics | Key Process Areas (KPAs) |
|---|---|---|
| 1 Initial | Ad hoc, chaotic, success depends on individuals | None |
| 2 Repeatable | Basic project management; earlier success can be repeated | Requirements management, project planning, project tracking, subcontract management, SQA, configuration management |
| 3 Defined | Standard documented process for the whole organisation | Process focus and definition, training programme, integrated software management, product engineering, inter-group coordination, peer reviews |
| 4 Managed | Process and product are measured quantitatively | Quantitative process management, software quality management |
| 5 Optimizing | Continuous improvement using feedback and new ideas | Defect prevention, technology change management, process change management |

### SEI-CMM vs ISO 9001

| Point | SEI-CMM | ISO 9001 |
|---|---|---|
| Origin | SEI, USA, for software | ISO, for all industries |
| Result | Grades organisation in 5 levels | Pass/fail certification |
| Focus | Continuous improvement of software process | Meeting a minimum quality standard |
| Approach | Specific to software, detailed KPAs | General guidelines |
| Validity | Assessment-based, no fixed term | Certificate renewed periodically |

## 13. Six Sigma

- Developed by Bill Smith at Motorola (1987); GE adopted it in 1995. A disciplined, **data-driven** method to reach near-perfect quality: **6 standard deviations from the mean = only 3.4 defects per million opportunities**.
- Three key elements: **Customer** (customers define quality), **Processes** (define and measure processes), **Employees** (trained Green Belts / Black Belts / Master Black Belts).
- Key concepts: critical to quality, defect, process capability, variation, stable operations, design for Six Sigma.

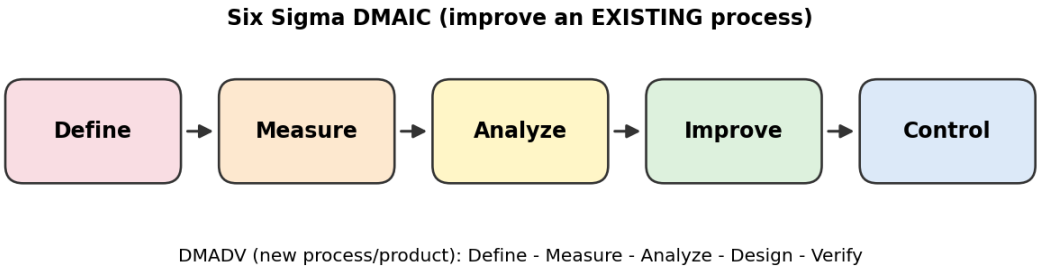



*Fig 19: Six Sigma methodologies*

- **DMAIC** (Define, Measure, Analyze, Improve, Control): improve an existing process. **DMADV** (Define, Measure, Analyze, Design, Verify): design a new process/product.

## Best practices of software engineering

- Develop iteratively; manage requirements; use component-based architectures; model visually; verify quality continuously; manage changes.

# UNIT IV (c): Computer-Aided Software Engineering (CASE)

## 1. Introduction and Definition

**CASE**
Computer-Aided Software Engineering is the use of **software tools to automate and support SDLC activities**: development, project management and maintenance. It integrates automated modelling tools into the development process.

- Goal: faster, better, cheaper software with fewer errors.
- CASE supports many methodologies: structured, object-oriented, RAD, prototyping, JAD.
- Includes IDEs, data dictionaries and tools to visualise existing artifacts.

**Example: CASE tools**
Visual Paradigm / StarUML / Rational Rose (UML modelling), MS Project (planning), Git (version control), Selenium (testing), Enterprise Architect (design + code generation), Lucidchart (diagrams).

## 2. Scope of CASE

- **Requirement engineering**, software design, model generation (data model, DFD), process description, user interface design, **code generation**, test-case generation, document generation, **reverse engineering**, configuration management, version control, project management, screen and report generation, project planning.

## 3. CASE Technology: Classification

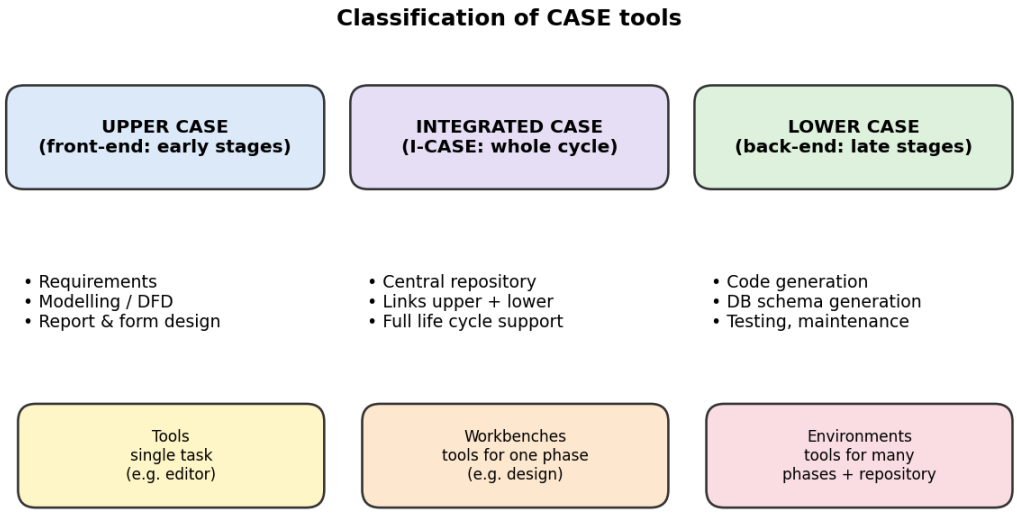



*Fig 20: Classification of CASE*

| Category | Meaning | Examples |
|---|---|---|
| Upper CASE | Supports early phases: planning, requirements, analysis, design | Report/form generators, modelling tools, DFD / UML editors |
| Lower CASE | Supports implementation and integration: coding, testing, maintenance | Code and database schema generators, debuggers, test tools |
| Integrated CASE (I-CASE) | Supports whole life cycle using a common repository | Tools sharing one repository |

### Tools, Workbenches, Environments

- **Tools:** support one specific task (editor, compiler, debugger).

- **Workbench:** integrated set of tools supporting **one phase** (design workbench, testing workbench). **Open workbench:** integration protocols open to all; **Closed workbench:** proprietary.
- **Environment:** collection of tools and workbenches supporting **many phases** around a repository.

## 4. Types of tools in a CASE environment

- **Central repository:** shared store of all project data (requirements, designs, code, test data).
- Project management and planning tools (schedule, cost and effort estimation).
- Analysis and design tools; prototyping tools (screen layouts, reports).
- Database management tools; integration and testing tools.
- Reengineering tools (migrate old to new platform); documentation tools; report generators.
- Software configuration management tools (version, change control, audit); quality assurance tools; web development tools; system software tools.

## 5. CASE Support in SDLC

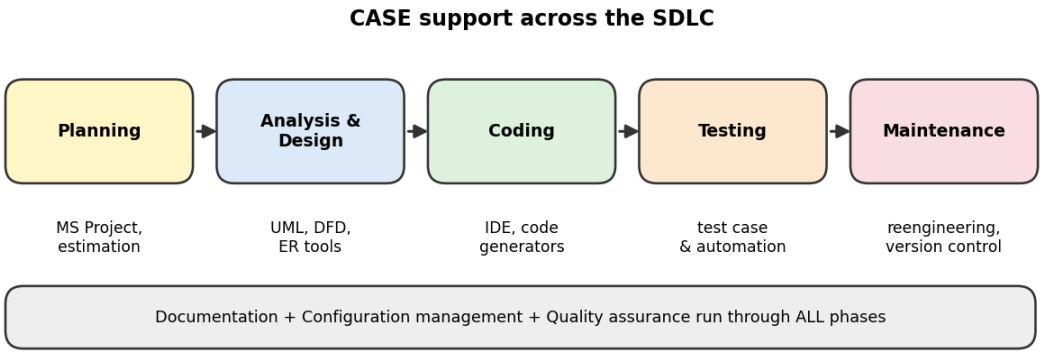



*Fig 21: CASE tools in different SDLC phases*

| SDLC phase | CASE support |
|---|---|
| Planning / project management | Scheduling, cost estimation, resource tracking, risk tracking |
| Prototyping | Screen and report layout, quick demo to user |
| Analysis and design | DFD, ER, UML editors, data dictionary, consistency checking |
| Coding | Code generators, editors, compilers, debuggers |
| Testing | Test data and test case generation, automated testing, coverage analysis |
| Maintenance and reengineering | Reverse engineering, version control, impact analysis |
| Documentation | Automatic generation of documents from the repository |

## 6. Characteristics, Advantages and Limitations

- **Characteristics of a good CASE tool:** follows a methodology, flexible, integrated, validation (consistency checks), supports collaboration, repository interface, testing support, reverse engineering, help system.

| Advantages | Limitations |
|---|---|
| Automates manual work; saves cost and time; improves quality and productivity; single design methodology; helps maintenance, requirement and configuration management; saves manual documentation; promotes reuse and standards; prototyping | Needs time to learn; purchase and training cost is high; limited documentation facilities in some tools; needs updating with technology; bad use at wrong places can result in poor design |

## 7. Second Generation CASE Tools

| First generation | Second generation |
|---|---|
| Prototyping tools, graphical design tools and code generators | More effective throughout development |
| Limited database support and access | Strong data modelling and repository / data dictionary |
| Gap between design and application generation | Bridges design and code: **application generation** |
| Isolated tools, little integration | Integrated tools, whole life-cycle automation |
| Limited diagrams | Supports structure charts, DFD, state charts, data models, OO diagrams; customisable; adequate documentation |

## 8. Architecture of a CASE Environment

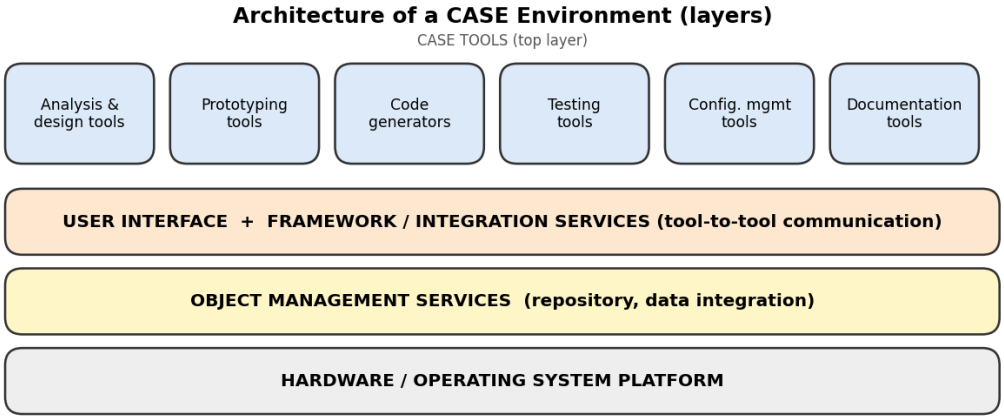



*Fig 22: Layered architecture of a CASE environment*

- **Hardware / operating system platform:** bottom layer where everything runs.
- **Object management services (repository):** stores and manages all project objects, versions and relations; gives data integration.
- **User interface and integration framework:** common look and feel; lets tools talk to each other (control and message integration).
- **CASE tools (top):** analysis, design, coding, testing, configuration management, documentation tools used by engineers.
- Integration happens at 3 levels: data (shared repository), control (tools call each other), and presentation (same UI).

**Exam tip**
For the CASE question draw the layered architecture figure, list upper/lower/integrated CASE, and mention the first vs second generation table.

# Likely 20-Mark Questions (Practice)

- Q1. What is software maintenance? Explain its categories and any two maintenance process models with diagrams.
- Q2. Explain Lehman's laws of software evolution. Why is software change inevitable?
- Q3. What is reengineering? Explain reverse engineering, restructuring and forward engineering with a neat diagram.
- Q4. Explain the reengineering process and its activities: planning, program comprehension, re-documentation.
- Q5. Calculate ACT and maintenance cost: development cost Rs 40 lakh, ACT 8%, 5 years. Explain SMI.
- Q6. What is software aging? Explain causes, symptoms and rejuvenation.
- Q7. Define dependability. Explain availability, reliability and safety and how they are related.
- Q8. Explain reliability metrics (POFOD, ROCOF, MTTF, MTTR, MTBF) with examples and reliability achievement approaches.
- Q9. Describe McCall's quality factors. Differentiate verification and validation.
- Q10. What is SQA? Explain SQA activities, ISO 9000 and the SEI-CMM levels.
- Q11. What is CASE? Explain classification, CASE support in SDLC and second generation CASE tools.
- Q12. Draw and explain the architecture of a CASE environment.

**Example: Q5 solution**
Total cost = 40 + 5 x (40 x 0.08) = 40 + 16 = **Rs 56 lakh**.

## One-page Quick Revision

| Topic | Remember |
|---|---|
| Maintenance types | Corrective, Adaptive, Perfective, Preventive (+ Emergency) |
| Models | Quick-fix, Osborne, Iterative enhancement, Full reuse, IEEE 1219 (7 phases), ISO 12207 (6 phases) |
| Metrics | ACT = (KLOC added + deleted)/KLOC total; Effort = ACT x Dev effort (x EAF); SMI = (Mt - (Fa+Fc+Fd))/Mt |
| Reengineering | Reverse engg + Δ + Forward engg; 6 activities: planning, comprehension, re-documentation, reverse, restructuring, forward |
| Aging | Causes: patching, complexity, no docs, leaks. Cure: design for change, rejuvenation |
| Dependability | Availability, Reliability, Safety, Security (+ maintainability, survivability) |
| Reliability metrics | POFOD, ROCOF, MTTF, MTTR, MTBF = MTTF + MTTR; Availability = MTTF/(MTTF+MTTR) |
| Reliability approaches | Fault avoidance, fault detection and removal, fault tolerance |
| Safety approaches | Hazard avoidance, hazard detection and removal, damage limitation |
| Quality factors | Operation (5), Revision (3), Transition (3) = 11 factors (McCall) |
| V&V; | Verification: building product right. Validation: building right product. |
| SQA | Preventive, planned and systematic; SQA plan, audits, reviews |
| ISO 9000 | Process certification; 8 principles; 6 certification steps |
| CMM | Initial, Repeatable, Defined, Managed, Optimizing |
| Six Sigma | 3.4 defects per million; DMAIC / DMADV |
| CASE | Upper, Lower, Integrated; tools -> workbenches -> environments; repository at centre |

**Best of luck for Test-2!**